

## Model

```
using System;
using System.Collections.Generic;

namespace SameGame.Models
{
    public enum GameMode
    {
        Easy,
        Medium,
        Hard
    }
    public enum Color
    {
        Red,
        Green,
        Blue,
        Orange,
        Pink,
        Yellow,
        Gray,
        None
    }
}

public class GameModel
{
    private Random _random = new Random();
    private Color[,] _board = new Color[15, 15];
    public GameMode Mode { get; set; } = GameMode.Easy;

    public Color[,] Board => _board;

    public event EventHandler? BoardChanged;

    public GameModel()
    {
        // A konstruktorban csak indítunk egy új játékot
        NewGame();
    }

    // Segédfüggvény, hogy ne kelljen mindig direktben a tömböt elérni
    public Color GetColor(int row, int col)
    {
        if (row < 0 || row >= 15 || col < 0 || col >= 15)
            return Color.None;
        return _board[row, col];
    }

    public void deleteColor(int row, int col)
    {
        // 1. Ellenőrzés: Pályán belül vagyunk-e?
        if (row < 0 || row >= 15 || col < 0 || col >= 15) return;

        // 2. Ellenőrzés: Ha már üres mezőre kattintottunk, ne történjen
        semmi
        if (_board[row, col] == Color.None) return;

        Color targetColor = _board[row, col];

        // Itt tároljuk, hogy mely mezőket kell törölni
        List<(int, int)> itemsToDelete = new List<(int, int)>();

        // Segéd tömb, hogy tudjuk, melyik mezőt néztük már meg (hogy
        ne kerüljünk végtelen ciklusba)
        bool[,] visited = new bool[15, 15];

        // Megkeressük az összes szomszédot
        SearchNeighbors(row, col, targetColor, itemsToDelete, visited);

        // Ha találtunk elemeket (önmagán kívül is, vagy csak önmagát),
        akkor töröljük őket
        // A szabály szerint általában klikk-re törölődik, akkor is ha egyed
        ül van.
        if (itemsToDelete.Count > 0)
        {
            foreach (var item in itemsToDelete)
            {
                _board[item.Item1, item.Item2] = Color.None;
            }
        }
    }
}
```

```
// Miután töröltünk, jöhet a "leesés"
ApplyGravity();

// Jelezzük a felületnek, hogy frissüljön
BoardChanged?.Invoke(this, EventArgs.Empty);
}

// Ez a rekurzív függvény keresi meg a szomszédokat
private void SearchNeighbors(int r, int c, Color target, List<(int, int)>
items, bool[,] visited)
{
    // Pálya szélének ellenőrzése
    if (r < 0 || r >= 15 || c < 0 || c >= 15) return;

    // Ha már láttuk ezt a mezőt, nem foglalkozunk vele
    if (visited[r, c]) return;

    // Ha nem olyan a színe, mint amit keresünk, megállunk
    if (_board[r, c] != target) return;

    // Megjelöljük, hogy itt jártunk, és hozzáadjuk a törlendők listájá
    hoz
    visited[r, c] = true;
    items.Add((r, c));

    // Megnézzük a négy irányt (Fel, Le, Balra, Jobbra)
    // Itt hívja meg önmagát a függvény (rekurzio)
    SearchNeighbors(r - 1, c, target, items, visited); // Fel
    SearchNeighbors(r + 1, c, target, items, visited); // Le
    SearchNeighbors(r, c - 1, target, items, visited); // Balra
    SearchNeighbors(r, c + 1, target, items, visited); // Jobbra
}

// Ez intézi, hogy a kockák leessenek a lyukak helyére
private void ApplyGravity()
{
    // Oszloponként megyünk végig (balról jobbra)
    for (int col = 0; col < 15; col++)
    {
        // Egy oszlopon belül lentől felfelé haladunk
        // 'writeRow' mutatja, hova kell leesnie a következő kockának
        int writeRow = 14;

        for (int row = 14; row >= 0; row--)
        {
            if (_board[row, col] != Color.None)
            {
                // Ha találunk egy kockát, azt "leírjuk" a writeRow
                pozícióba
                _board[writeRow, col] = _board[row, col];

                // Ha a writeRow nem egyezik a row-val (tehát esett a
                kocka),
                // akkor az eredeti helyét töröljük (bár a következő körben
                úgys felülíródhat, de a tetején fontos)
                if (writeRow != row)
                {
                    _board[row, col] = Color.None;
                }

                // A következő kocka eggyel feljebb fog érkezni
                writeRow--;
            }
        }

        // A maradék helyeket a tetején feltöltjük üressel (biztonság
        kedvéért)
        while (writeRow >= 0)
        {
            _board[writeRow, col] = Color.None;
            writeRow--;
        }
    }
}
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> public void NewGame() {     _board = new Color[15, 15];      // Meghatározzuk hány szín legyen a nehézség alapján     int colorCount = 3; // Easy alapértelmezett     if (Mode == GameMode.Medium) colorCount = 5;     if (Mode == GameMode.Hard) colorCount = 7;      for (int i = 0; i &lt; 15; i++)     {         for (int j = 0; j &lt; 15; j++)         {             // Véletlen szám generálása a színek számának megfelelően             int rand = _random.Next(colorCount);              // Egyszerű hozzárendelés             if (rand == 0) _board[i, j] = Color.Red;             else if (rand == 1) _board[i, j] = Color.Green;             else if (rand == 2) _board[i, j] = Color.Blue;             else if (rand == 3) _board[i, j] = Color.Orange;             else if (rand == 4) _board[i, j] = Color.Pink;             else if (rand == 5) _board[i, j] = Color.Yellow;             else if (rand == 6) _board[i, j] = Color.Gray;         }     }      // Értesítjük a nézetet, hogy új pálya van     BoardChanged?.Invoke(this, EventArgs.Empty); }  public void ChangeMode(GameMode mode) {     Mode = mode;     NewGame(); } } </pre> | <pre> <b>FieldVM</b> using CommunityToolkit.Mvvm.Input; using SameGame.Models; using System;  namespace SameGame.ViewModels {     public class FieldVM : ViewModelBase     {         private readonly GameModel _model;          public int Row;         public int Col;          public FieldVM (GameModel model, int row, int col)         {             _model = model;             Row = row;             Col = col;         }          public string BackGround =&gt; _model.Board[Row, Col] switch         {             Color.Red =&gt; "Red",             Color.Blue =&gt; "Blue",             Color.Gray =&gt; "Gray",             Color.Pink =&gt; "Pink",             Color.Green =&gt; "Green",             Color.Yellow =&gt; "Yellow",             Color.Orange =&gt; "Orange",             _ =&gt; "White"         };          public void Refresh()         {             OnPropertyChanged(nameof(BackGround));         }          public RelayCommand? ClickCommand { get; set; }     } } </pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## ViewModel

```
using CommunityToolkit.Mvvm.Input;
using SameGame.Models;
using System;
using System.Collections.ObjectModel;

namespace SameGame.ViewModels
{
    public partial class MainWindowViewModel : ViewModelBase
    {
        private readonly GameModel _model;

        // Parancs az új játékhoz
        public RelayCommand NewGameCommand { get; set; }

        // Új parancs a nehézség váltáshoz (string paramétert vár)
        public RelayCommand<string> ChangeModeCommand { get; set; }

        public ObservableCollection<FieldVM> Fields { get; set; }
        public event EventHandler? BoardChanged;
        public event EventHandler? NewGame;

        public MainWindowViewModel(GameModel model)
        {
            _model = model;
            // Ha a modellben változik a tábla (pl. törlés vagy új játék miatt),
            frissítjük a nézetet
            _model.BoardChanged += (s, e) => UpdateView();

            Fields = new ObservableCollection<FieldVM>();
            for (int row = 0; row < 15; row++)
            {
                for (int col = 0; col < 15; col++)
                {
                    var field = new FieldVM(_model, row, col);
                    field.ClickCommand = new RelayCommand(() =>
OnFieldClicked(field));
                    Fields.Add(field);
                }
            }

            NewGameCommand = new RelayCommand(OnNewGame);

            ChangeModeCommand = new
RelayCommand<string>(OnChangeMode);

            _model.NewGame();
        }

        private void OnFieldClicked(FieldVM field)
        {
            _model.DeleteColor(field.Row, field.Col);
            // Az UpdateView hívása automatikus a BoardChanged esemény
            miatt
        }

        // Ez fut le, ha valamelyik nehézségi gombra kattintanak
        private void OnChangeMode(string? modeStr)
        {
            // Megpróbáljuk a szöveget (pl. "Hard") átalakítani GameMode
            típusá
            if (Enum.TryParse(modeStr, out GameMode mode))
            {
                _model.ChangeMode(mode);
            }
        }

        public void UpdateView()
        {
            foreach (var field in Fields)
            {
                field.Refresh();
            }
        }

        private void OnNewGame()
        {
            _model.NewGame();
        }
    }
}
```

## MainWindow

```
<Grid RowDefinitions="Auto, *">

    <StackPanel Grid.Row="0" Orientation="Horizontal" Margin="10"
Spacing="10">
        <Button Content="Új játék" Command="{Binding
NewGameCommand}" Background="LightGray"/>

        <TextBlock Text="Nehézség:" VerticalAlignment="Center"
Margin="20,0,5,0" FontWeight="Bold"/>

        <Button Content="Könnyű"
Command="{Binding ChangeModeCommand}"
CommandParameter="Easy"/>

        <Button Content="Közepes"
Command="{Binding ChangeModeCommand}"
CommandParameter="Medium"/>

        <Button Content="Nehéz"
Command="{Binding ChangeModeCommand}"
CommandParameter="Hard"/>
    </StackPanel>

    <ItemsControl Grid.Row="1" ItemsSource="{Binding Fields}"
Margin="10">
        <ItemsControl.ItemsPanel>
            <ItemsPanelTemplate>
                <UniformGrid Rows="15" Columns="15"></UniformGrid>
            </ItemsPanelTemplate>
        </ItemsControl.ItemsPanel>

        <ItemsControl.ItemTemplate>
            <DataTemplate>
                <Button Command="{Binding ClickCommand}"
Background="{Binding Background}"
HorizontalAlignment="Stretch"
VerticalAlignment="Stretch"
HorizontalContentAlignment="Center"
VerticalContentAlignment="Center"
BorderThickness="1"
BorderBrush="#33000000"
CornerRadius="0">
                    <TextBlock Text=""/>
                </Button>
            </DataTemplate>
        </ItemsControl.ItemTemplate>
    </ItemsControl>
</Grid>
```

## App

```
namespace SameGame
{
    public partial class App : Application
    {
        public override void Initialize()
        {
            AvaloniaXamlLoader.Load(this);
        }

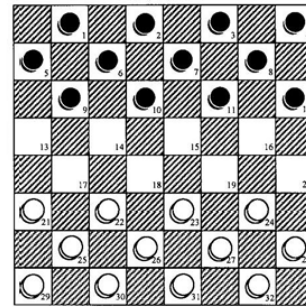
        public override void OnFrameworkInitializationCompleted()
        {
            if (ApplicationLifetime is IClassicDesktopStyleApplicationLifetime
desktop)
            {
                var model = new GameModel();
                var ViewModel = new MainWindowViewModel(model);

                desktop.MainWindow = new MainWindow
                {
                    DataContext = ViewModel,
                };
            }

            base.OnFrameworkInitializationCompleted();
        }
    }
}
```

## Feladat: Dáma

Készítsünk egy *Windows Presentation Foundation (WPF)* grafikus felületű, *MVVM* architektúrájú alkalmazást *C#* nyelven, amellyel a klasszikus Dáma játékot lehet játszani (egyszerűsített szabályokkal). A Dáma játékban két játékos (most fehér és fekete) játszanak egymás ellen, mindkettő három sor figurával (lásd ábra). A figurák egy mezőt léphetnek előre átlósan, vagy átugorhatnak egy ellenséges figurát (szintén átlósan), ha az ugrás irányában a figura mögött nincsen egy másik figura. Ezt hívják „ütésnek”, és ekkor a leütött figurát leszedjük a játéktérrel.



## Részfeladatok:

2. A program jelenítsen meg egy 8 x 8-as játéktáblát és helyezze rá az ábrán látható figurákat (ezt lehet szöveggel, színnel vagy képpel is jelezni). Legyen lehetőség felváltva lépésre (az ütés is lépésnek számít) és jelezzük különböző hibaüzenetekkel, ha a felhasználó egy nem szabályos lépést szeretne elvégezni. Ha az egyik félnek nincs több figurája, vagy nincs több szabályos lépése, akkor veszített. Ekkor legyen vége a játéknak és ezt felugró ablakkal jelezzük.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <p><b>Model</b></p> <pre>public enum Player { White, Black } public enum Piece { None, White, Black }  public class DamaModel {     public Piece[,] Board { get; private set; }     public Player CurrentPlayer { get; private set; }     public int Size { get; } = 8;      public event EventHandler&lt;string&gt;? GameOver;     public event EventHandler&lt;string&gt;? MoveError;      public DamaModel()     {         Board = new Piece[Size, Size];         NewGame();     }      public void NewGame()     {         // Tábla alaphelyzetbe állítása         for (int i = 0; i &lt; Size; i++)         {             for (int j = 0; j &lt; Size; j++)             {                 Board[i, j] = Piece.None;                 // Fekete bábuk a felső 3 sorban (0,1,2), csak a sötét mezőkön                 if (i &lt; 3 &amp;&amp; (i + j) % 2 != 0) Board[i, j] = Piece.Black;                 // Fehér bábuk az alsó 3 sorban (5,6,7)                 if (i &gt; 4 &amp;&amp; (i + j) % 2 != 0) Board[i, j] = Piece.White;             }         }         CurrentPlayer = Player.White;     }      public void Step(int fromX, int fromY, int toX, int toY)     {         // 1. Ellenőrizzük, hogy szabályos-e a lépés         // Az "out bool isCapture" megmondja, hogy történt-e ütés         if (!IsValidMove(fromX, fromY, toX, toY, out bool isCapture))         {             MoveError?.Invoke(this, "Szabálytalan lépés!");             return; // Ha hibás, kilépünk, nem történik semmi         }          // 2. Lépés végrehajtása: A célmezőre tesszük a bábút, a régijt töröljük         Board[toX, toY] = Board[fromX, fromY];         Board[fromX, fromY] = Piece.None;          // 3. Ha ütés történt, levesszük a köztes bábút         if (isCapture)         {             int midX = (fromX + toX) / 2; // A két mező közötti sor             int midY = (fromY + toY) / 2; // A két mező közötti oszlop             Board[midX, midY] = Piece.None;         }          // 4. Játékos váltás (Fehér -&gt; Fekete -&gt; Fehér)         CurrentPlayer = CurrentPlayer == Player.White ? Player.Black : Player.White;          // 5. Megnézzük, vége van-e a játéknak az új állapotban         CheckGameOver();     }      private bool IsValidMove(int x1, int y1, int x2, int y2, out bool isCapture)     {         isCapture = false;          // Pályán kívüli kattintás szűrése         if (x1 &lt; 0    x1 &gt;= Size    y1 &lt; 0    y1 &gt;= Size) return false;         if (x2 &lt; 0    x2 &gt;= Size    y2 &lt; 0    y2 &gt;= Size) return false;          // Csak a saját bábunkkal léphetünk         Piece currentPiece = Board[x1, y1];         if (currentPiece == Piece.None) return false;         if (CurrentPlayer == Player.White &amp;&amp; currentPiece != Piece.White) return false;         if (CurrentPlayer == Player.Black &amp;&amp; currentPiece != Piece.Black) return false;          // A célmezőnek üresnek kell lennie         if (Board[x2, y2] != Piece.None) return false;          int dx = x2 - x1; // Sorok különbsége         int dy = y2 - y1; // Oszlopok különbsége          // Irány ellenőrzése: Fehér "felfelé" (csökkenő sorindex), Fekete "lefelé" (növekvő sorindex) léphet         if (CurrentPlayer == Player.White &amp;&amp; dx &gt; 0) return false;         if (CurrentPlayer == Player.Black &amp;&amp; dx &lt; 0) return false;          // ESET A: Sima lépés (1-et lép átlósan)         if (Math.Abs(dx) == 1 &amp;&amp; Math.Abs(dy) == 1) return true;     } }</pre> |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|

```
// ESET B: Útés (2-t lép átlósan)
if (Math.Abs(dx) == 2 && Math.Abs(dy) == 2)
{
    int midX = (x1 + x2) / 2;
    int midY = (y1 + y2) / 2;
    Piece midPiece = Board[midX, midY];

    // Csak akkor üthetünk, ha van ott valami, és az az ellenfél
    if (midPiece == Piece.None) return false;
    if (CurrentPlayer == Player.White && midPiece == Piece.White)
return false;
    if (CurrentPlayer == Player.Black && midPiece == Piece.Black)
return false;

    isCapture = true;
    return true;
}

return false;
}

private void CheckGameOver()
{
    // Megszámoljuk, kinek hány bábuja maradt
    int currentPieces = 0;
    foreach (var p in Board)
    {
        if ((CurrentPlayer == Player.White && p == Piece.White) ||
            (CurrentPlayer == Player.Black && p == Piece.Black))
        {
            currentPieces++;
        }
    }

    string winner = CurrentPlayer == Player.White ? "Fekete" : "Fehér";

    // 1. feltétel: Elkfogytak a bábuk
    if (currentPieces == 0)
    {
        GameOver?.Invoke(this, $"A {winner} játékos győzött (elfogytak a
bábuk)!");
        return;
    }

    // 2. feltétel: Beszorult a játékos (nincs érvényes lépés)
    if (!IsValidMoves(CurrentPlayer))
    {
        GameOver?.Invoke(this, $"A {winner} játékos győzött (nincs érvé
nyes lépés)!");
    }
}

private bool IsValidMoves(Player player)
{
    for (int x = 0; x < Size; x++)
    {
        for (int y = 0; y < Size; y++)
        {
            // Ha a mi bábunkat találtuk meg...
            Piece p = Board[x, y];
            if ((player == Player.White && p == Piece.White) ||
                (player == Player.Black && p == Piece.Black))
            {
                // ...megpróbáljuk mind a 4 irányt (sima lépés és útés)
                int[] offsets = { -1, 1, -2, 2 };
                foreach (int dx in offsets)
                {
                    foreach (int dy in new[] { -1, 1, -2, 2 })
                    {
                        if (Math.Abs(dx) != Math.Abs(dy)) continue; // Csak
átlósan

                        // Ha találunk akár EGY érvényes lépést, akkor még
nincs vége

                        if (IsValidMove(x, y, x + dx, y + dy, out _))
                            return true;
                    }
                }
            }
        }
    }
    return false;
}
}
```

```
FieldVM
private string _text = "";
private string _color = "Transparent";

public int X { get; set; }
public int Y { get; set; }
public DelegateCommand? ClickCommand { get; set; }

// A bábu jele ("O" vagy üres)
public string Text
{
    get => _text;
    set { _text = value; OnPropertyChanged(); }
}

// Ez jelzi a kiválasztást (pl. piros keret vagy háttér)
public string Color
{
    get => _color;
    set { _color = value; OnPropertyChanged(); }
}

ViewModel
private readonly DamaModel _model;
// Itt tároljuk, melyik mező van épp kijelölve
private DamaField? _selectedField;

public ObservableCollection<DamaField> Fields { get; set; }

private string _status = "Következik: Fehér";
public string Status
{
    get => _status;
    set { _status = value; OnPropertyChanged(); }
}

public DamaViewModel(DamaModel model)
{
    _model = model;
    Fields = new ObservableCollection<DamaField>();

    for (int i = 0; i < 8; i++)
    {
        for (int j = 0; j < 8; j++)
        {
            Fields.Add(new DamaField
            {
                X = i,
                Y = j,
                // Minden mezőnek saját parancsa van, ami meghívja az
OnFieldClick-et
                ClickCommand = new DelegateCommand(param =>
OnFieldClick(param as DamaField))
            });
        }
    }
    RefreshTable(); // Kezdőállapot kirajzolása
}

private void RefreshTable()
{
    foreach (var field in Fields)
    {
        Piece p = _model.Board[field.X, field.Y];
        field.Text = p == Piece.None ? "" : "O"; // Ha van bábu, kiírjuk

        // Színek beállítása
        if (p == Piece.White) field.Color = "White";
        else if (p == Piece.Black) field.Color = "Black";
        else field.Color = "Transparent";
    }
    // Kiírjuk, ki következik
    Status = $"Következik: {(_model.CurrentPlayer == Player.White ?
"Fehér" : "Fekete")}";
}
}
```

```

private void OnFieldClick(DamaField? clickedField)
{
    if (clickedField == null) return;

    // 1. ÁLLAPOT: Még nincs kijelölve semmi -> Kijelölés
    if (_selectedField == null)
    {
        Piece p = _model.Board[clickedField.X, clickedField.Y];
        // Csak akkor jelöljük ki, ha a játékos a saját bábujaára kattintott
        if ((_model.CurrentPlayer == Player.White && p == Piece.White) ||
            (_model.CurrentPlayer == Player.Black && p == Piece.Black))
        {
            _selectedField = clickedField;
            _selectedField.Color = "Red"; // Pirosra színezzük
        }
    }
    // 2. ÁLLAPOT: Már van kijelölt bábu -> Lépés próba
    else
    {
        // Ha ugyanarra kattintunk megint, visszavonjuk a kijelölést
        if (_selectedField == clickedField)
        {
            RefreshTable(); // Visszaállítjuk a színeket
            _selectedField = null;
        }
        else
        {
            // Megpróbáljuk a lépést a Modelben
            // Ha szabálytalan, a Model eseményt küld az App-nak
            (hibaüzenet)
            _model.Step(_selectedField.X, _selectedField.Y, clickedField.X,
            clickedField.Y);

            // Akár sikerült, akár nem, töröljük a kijelölést és frissítjük a táblát
            _selectedField = null;
            RefreshTable();
        }
    }
}

```

## App

```

public App()
{
    this.Startup += App_Startup;
}

private void App_Startup(object? sender, StartupEventArgs e)
{
    // 1. Model létrehozása
    DamaModel _model = new DamaModel();

    // 2. Feliratkozás a Model eseményeire közvetlenül
    _model.GameOver += Model_GameOver;
    _model.MoveError += Model_MoveError;

    // 3. ViewModel létrehozása (átadjuk neki a Modelt)
    DamaViewModel _viewModel = new DamaViewModel(_model);

    // 4. View létrehozása
    MainWindow _view = new MainWindow();
    _view.DataContext = _viewModel;
    _view.Show();
}

private void Model_MoveError(object? sender, string message)
{
    MessageBox.Show(message, "Hiba", MessageBoxButton.OK,
    MessageBoxImage.Warning);
}

private void Model_GameOver(object? sender, string message)
{
    MessageBox.Show(message, "Játék vége", MessageBoxButton.OK,
    MessageBoxImage.Information);
}

```

## MainWindow

```

<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <TextBlock Text="{Binding Status}" FontSize="20"
    HorizontalAlignment="Center" Margin="10"/>

    <ItemsControl Grid.Row="1" ItemsSource="{Binding Fields}">
        <ItemsControl.ItemsPanel>
            <ItemsPanelTemplate>
                <UniformGrid Rows="8" Columns="8"/>
            </ItemsPanelTemplate>
        </ItemsControl.ItemsPanel>
        <ItemsControl.ItemTemplate>
            <DataTemplate>
                <Button Command="{Binding ClickCommand}"
                CommandParameter="{Binding}" Margin="1">
                    <Button.Background>
                        <SolidColorBrush Color="LightGray"/>
                    </Button.Background>

                    <Grid>
                        <Ellipse Width="30" Height="30" Fill="{Binding Color}"
                        Stroke="Gray"/>
                    </Grid>
                </Button>
            </DataTemplate>
        </ItemsControl.ItemTemplate>
    </ItemsControl>
</Grid>

```